Course: Cloud and Network Security CNS-C2-2026

Student Name: James Kirika
Student No: CS-CNS12-26028

October 10, 2026

Week 5 Assignment 1:
Introduction to web application

Introduction 3
Web Application Architectures 3
Front-end vs. Back-end 3
Front-end Components 3
HTML 3
CSS 3
JavaScript 3
Front-end Vulnerabilities 4
Data Exposure 4
Exercise: 4
HTML Injection 4
Exercise: 5
XSS / CSRF 5
Exercise: 6
Back-end Components 6
Back-end Servers 6
Web Servers 6
Databases 6
Development Frameworks & APIs 6
Exercise: 7
Back-end Vulnerabilities 7
Public Vulnerabilities 7
Exercise: 8
Common Web Vulnerabilities 8
Web Application Security Assessment 9
1. Executive Summary 9
2. Methodology 9
3. Findings 9
3.1 SQL Injection in /api/greet (INSERT statement) 9
3.2 Server Version Disclosure 11
3.3 State-Changing Logic Exposed via HTTP GET 12
4. Overall Risk Rating 13
5. Recommendations 13
6. Appendix: Environment 13
Conclusion 13
Lab completions proof 14
Links 14

Introduction

Web applications are interactive programs that run in a browser and follow a client-server architecture: front-end components (the interface the user sees) run client-side in the browser, while back-end components (application source code, servers, databases) run server-side. This lets organizations host powerful, centrally controlled applications that are accessible worldwide, with Gmail, Amazon, and Google Docs as typical examples.

Web Application Architectures

A web application architecture defines how the front end, back end, and database interact to serve a request. The most common pattern is a multi-tier (client-server) architecture: the browser sends an HTTP request, a web server routes it to application logic, and that logic queries a database before returning a response. More complex applications split this further into microservices, where individual functions (authentication, payments, search) run as separate, independently deployable services communicating over APIs. The architecture chosen affects not just performance and scalability but also the attack surface, since each additional tier, service, or API is a potential entry point for an attacker.

Front-end vs. Back-end

The front end is everything that executes in the user's browser: the HTML structure, CSS styling, and JavaScript behaviour that the user directly sees and interacts with. The back end is everything that runs on the server, out of the user's direct view: application logic, authentication, business rules, and database access. The practical security distinction is trust. Front-end code is fully visible and modifiable by the user (via browser DevTools, for instance), so it can never be relied on to enforce security; all real validation and authorization must happen on the back end, which the attacker cannot directly inspect or alter.

Front-end Components

HTML

HTML (HyperText Markup Language) defines the structure and content of a web page: headings, paragraphs, forms, links, and the elements a browser renders. It carries no logic of its own, but it is the layer where user input is captured (via forms and input fields) and where injected content, if not properly handled elsewhere, ends up being rendered to the page.

CSS

CSS (Cascading Style Sheets) controls the visual presentation of HTML: layout, color, typography, and responsiveness across devices. CSS is largely a cosmetic layer, but it has its own minor attack surface (for example, CSS-based data exfiltration via attribute selectors), and it is frequently manipulated alongside HTML injection to make malicious content visually convincing.

JavaScript

JavaScript is the scripting language that runs in the browser and gives pages interactivity: handling clicks, validating forms client-side, and making asynchronous calls to back-end APIs (as with the fetch() calls used in the accompanying lab). Because JavaScript executes in the user's browser and can manipulate

the page's DOM freely, it is also the vector through which most client-side attacks, particularly Cross-Site Scripting, achieve their effect.

Front-end Vulnerabilities

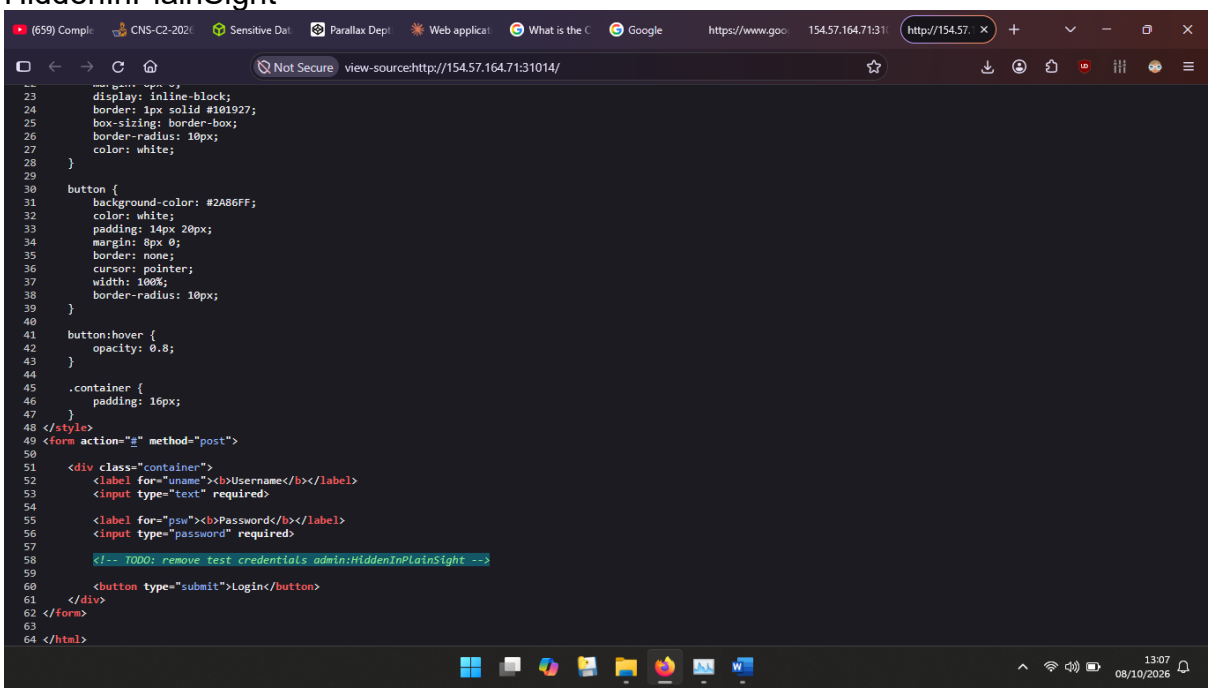
Data Exposure

Data exposure occurs when sensitive information is unintentionally made available to the client: hardcoded API keys or credentials in JavaScript source, verbose comments left in HTML, debug information in response headers, or overly detailed error messages. Because all front-end code and network traffic is visible to the user via View Source or browser DevTools, anything sent to the browser must be treated as public, regardless of whether a link to it is displayed anywhere in the UI.

Exercise:

Check the above login form for exposed passwords. Submit the password as the answer.

HiddenInPlainSight

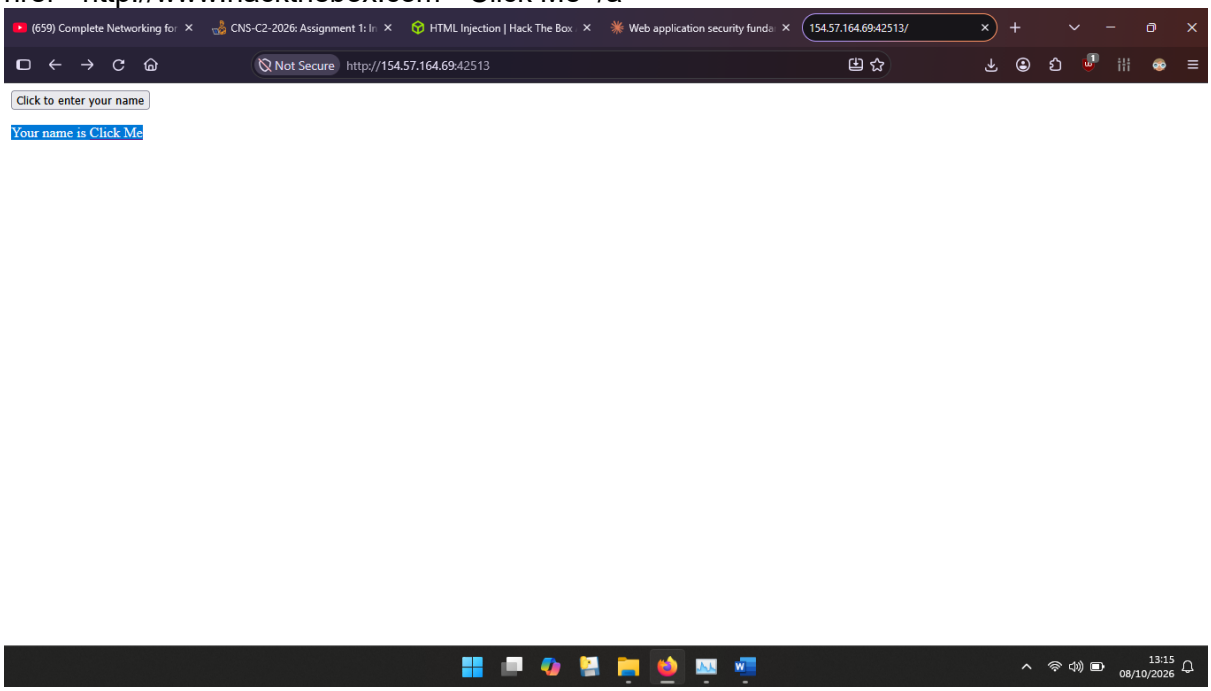


HTML Injection

HTML injection happens when user-supplied input is inserted into a page's HTML without sanitization, letting an attacker alter the page's structure and content. In its mildest form this defaces a page (injected headings, fake forms); combined with JavaScript it becomes a vector for session theft, credential phishing, or redirecting users to malicious sites, which is why proper output encoding of user input is required wherever it is rendered back into HTML.

Exercise:

What text would be displayed on the page if we use the following payload as our input: Click Me

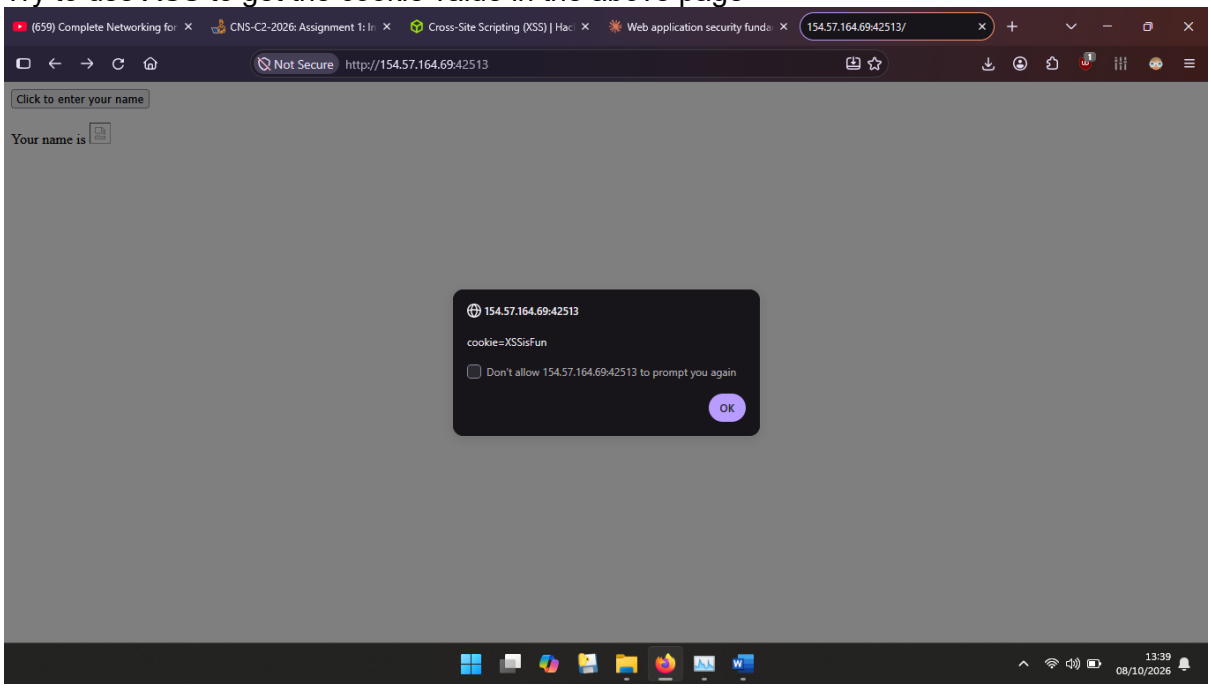


XSS / CSRF

Cross-Site Scripting (XSS) is a specific, more severe form of injection in which an attacker gets arbitrary JavaScript to execute in another user's browser, within the trusted context of the vulnerable site. This can be used to steal session cookies, log keystrokes, or perform actions as the victim. Cross-Site Request Forgery (CSRF) is a related but distinct issue: it tricks a logged-in user's browser into submitting a request to a site they're authenticated to, without their knowledge, which is exactly the class of flaw identified and fixed in the accompanying lab when a GET request was found to trigger a database write.

Exercise:

Try to use XSS to get the cookie value in the above page



Back-end Components

Back-end Servers

The back-end server is the machine (physical or virtual) that runs the application logic, processes requests, and communicates with the database. It hosts the runtime environment, in the lab's case a Python interpreter running a Flask application, and is the component ultimately responsible for enforcing business rules and access control.

Web Servers

A web server (such as Apache, Nginx, or, in development, Werkzeug) is the software that listens for incoming HTTP requests, serves static files, and routes dynamic requests to the application code. Web servers are also a common source of their own vulnerabilities, misconfigurations, outdated software versions, and default credentials, which is why version fingerprinting, as demonstrated against the lab's Server header, is a standard reconnaissance step in web assessments.

Databases

The database stores the application's persistent data; relational systems (MySQL, PostgreSQL, SQLite) and non-relational systems (MongoDB, Redis) are both common. Because databases hold the data attackers usually want, the interface between application code and the database, specifically, how user input is incorporated into queries, is one of the highest-value targets in a web application, as demonstrated by the SQL injection found in the accompanying lab.

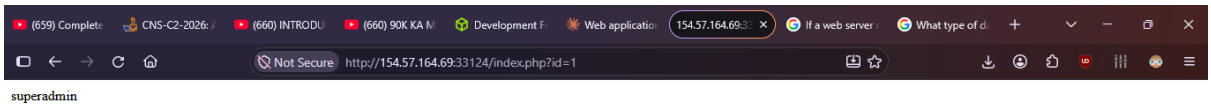
Development Frameworks & APIs

Frameworks (Flask, Django, Express, Laravel, and similar) provide the scaffolding that back-end developers build on: routing, templating, session handling, and database abstractions. APIs (Application Programming Interfaces) define how different components, or different applications entirely,

communicate, typically over HTTP using REST or similar conventions. Frameworks speed up development and often include built-in protections (e.g. parameterized query helpers, CSRF tokens), but only when developers use them correctly; the lab's vulnerabilities were introduced specifically by bypassing Flask's safer defaults in favor of raw string concatenation and an unrestricted HTTP method.

Exercise:

Use GET request '/index.php?id=0' to search for the name of the user with id number 1?



superadmin



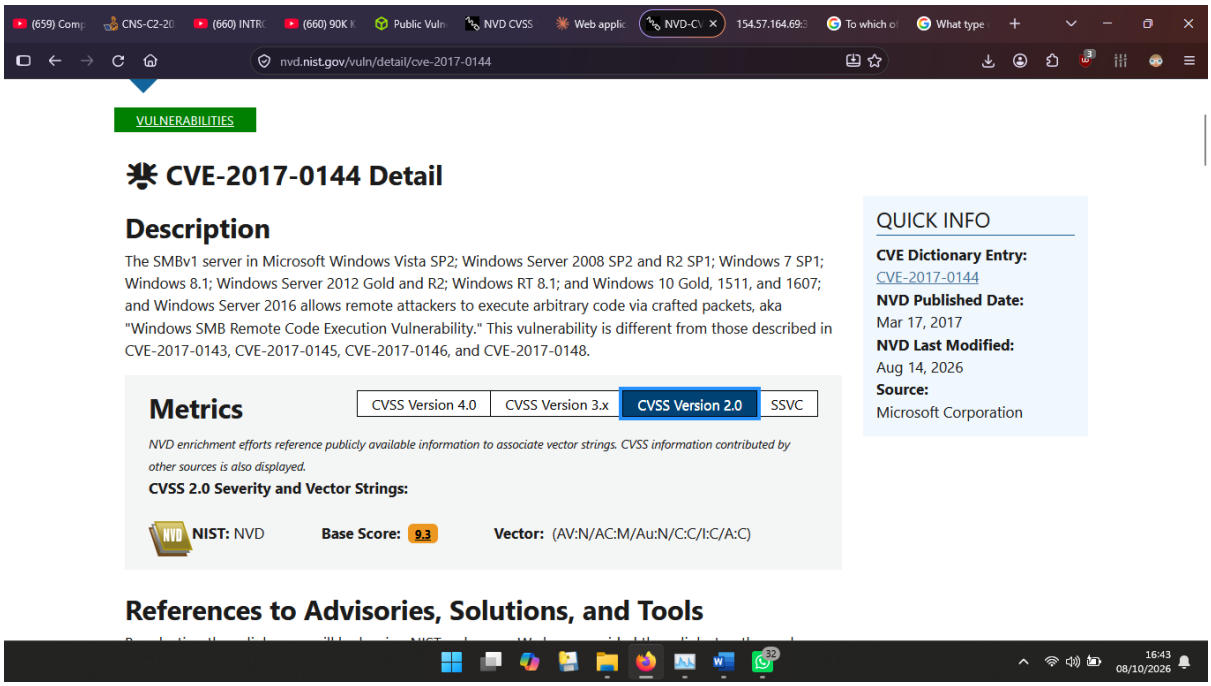
Back-end Vulnerabilities

Public Vulnerabilities

Public vulnerabilities are documented, known weaknesses in specific software versions, cataloged under identifiers such as CVEs (Common Vulnerabilities and Exposures) and scored for severity using CVSS. Identifying the exact software and version in use, as with CVE-2017-0144 researched earlier in this module, allows both defenders and attackers to check it against public vulnerability databases; this is precisely why unnecessary version disclosure (as found and remediated in the lab's Server header) is a risk, since it shortens that lookup for an attacker.

Exercise:

What is the CVSS v2.0 score of the public vulnerability CVE-2017-0144? 9.3



Common Web Vulnerabilities

Beyond named CVEs, most back-end compromises come from a recurring set of implementation flaws, commonly organized around lists like the OWASP Top 10: injection flaws (SQL, command, and others), broken authentication and session management, insecure direct object references, security misconfiguration, and insufficient input validation. The lab exercise accompanying this module demonstrated this directly: a SQL injection vulnerability was introduced through string-concatenated queries, confirmed by breaking the query with a single quote and then inserting an unauthorized database row, and remediated using parameterized queries, the standard fix for this entire vulnerability class.

Web Application Security Assessment

Self-Directed Lab — HTB Academy: Introduction to Web Applications

Target	Flask + SQLite web application (localhost:8080)
Tester	Jim
Date	October 2026
Scope	Custom-built lab application: index.html, script.js, app.py, lab.db
Objective	Build a small web app end-to-end, then test and remediate it for common vulnerabilities per OWASP-style methodology

1. Executive Summary

This assessment was performed against a self-built web application created to practice the full web application lifecycle: front-end development, back-end development, database integration, API design, vulnerability testing, and remediation. The application is a minimal visitor-greeting tool backed by a SQLite database, served by a Python Flask backend.

Two findings were identified during testing: a SQL Injection vulnerability in the visitor-insert logic (High severity), and a Server Version Disclosure issue via HTTP response headers (Low severity). An additional insecure design pattern — state-changing logic reachable via HTTP GET — was identified and corrected proactively. All findings were remediated and re-tested to confirm the fixes were effective.

- SQL Injection in /api/greet (INSERT statement) — **High** | Remediated | CVSS-style: 8.1
- Server version disclosure (Werkzeug/Python in headers) — **Low** | Remediated | CVSS-style: 3.7
- State-changing logic exposed via HTTP GET (/api/visitors) — **Low** | Remediated | CVSS-style: 3.1

2. Methodology

Testing followed a build-then-break-then-fix approach: the application was first built out fully (front end, back end, database, API), then attacked by hand from the command line using curl, and finally patched with standard remediations. This mirrors the OWASP Testing Guide's black-box approach to input validation and API testing, applied here in a white-box context since the tester also wrote the application.

- Front end: HTML, CSS, JavaScript (fetch-based API calls)
- Back end: Python 3.14, Flask (Werkzeug development server)
- Database: SQLite 3
- Attack tooling: curl.exe (Windows), browser DevTools
- Network exposure: application bound to 127.0.0.1 only — no external network exposure during testing

3. Findings

3.1 SQL Injection in /api/greet (INSERT statement)

Severity: High CVSS-style score: 8.1

Description:

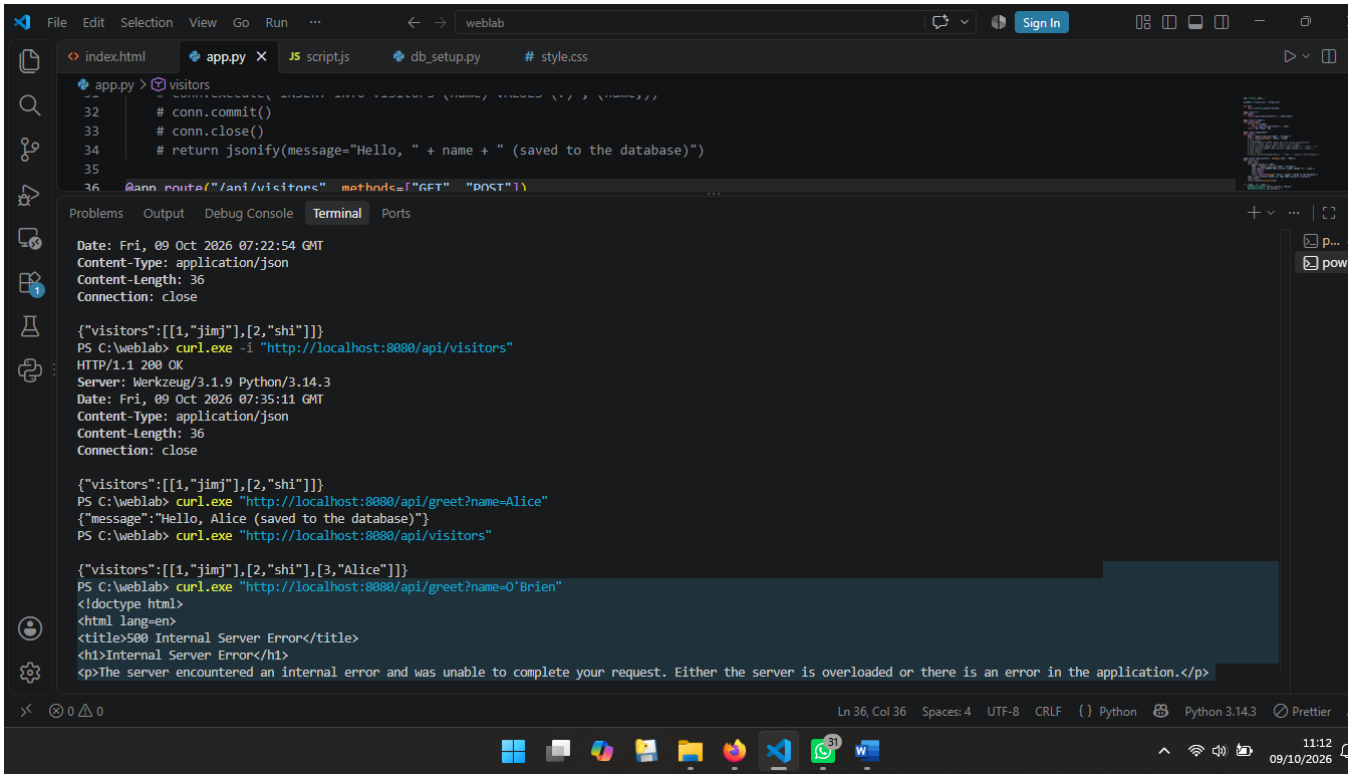
The /api/greet endpoint accepted a user-supplied “name” parameter and inserted it into the visitors table using direct string concatenation rather than a parameterized query. This allowed an attacker to break out of the intended string literal and inject arbitrary SQL.

Vulnerable code:

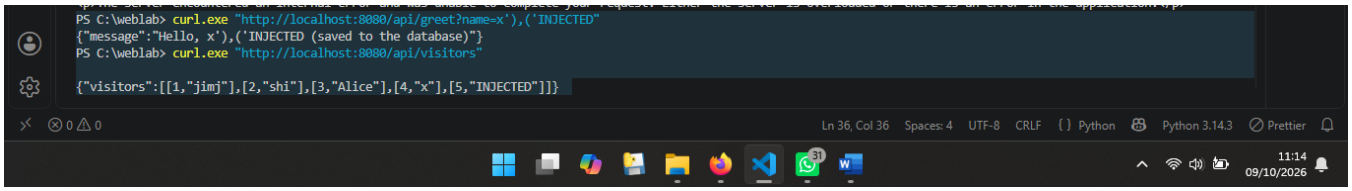
```
conn.execute("INSERT INTO visitors (name) VALUES ('" + name + "')")
```

Proof of Concept:

1. A single quote in the name parameter caused an unhandled server error, confirming the field was not sanitized:



2. Closing the VALUES clause and opening a second one allowed an attacker-controlled row to be inserted that was never intended by the application logic:



Impact:

In this lab application the impact is limited to extra rows in a single table. In a production application with the same pattern, this class of flaw routinely leads to unauthorized data insertion, data exfiltration via UNION-based injection, authentication bypass, or full database compromise, depending on the privileges of the database account in use.

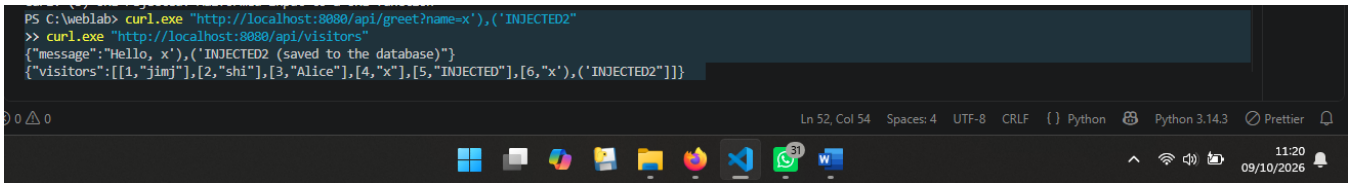
Remediation:

Replaced string concatenation with a parameterized query, letting the database driver handle escaping:

```
conn.execute("INSERT INTO visitors (name) VALUES (?)", (name,))
```

Re-test:

The same payload was resubmitted after the fix. It was stored as a single literal value with no second row created and no server error:

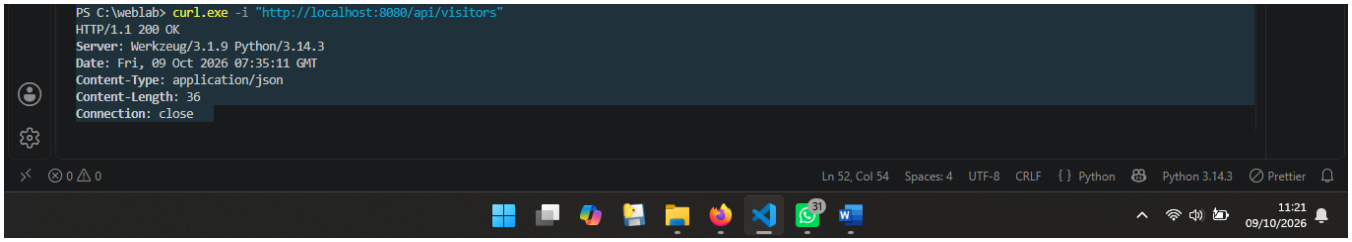


3.2 Server Version Disclosure

Severity: Low CVSS-style score: 3.7

Description:

The default Werkzeug development server returned its exact software name and version, and the Python interpreter version, in the Server HTTP response header. This gives an attacker a precise fingerprint to match against known CVEs for that framework and language version, the same technique used in Section 4 of the related HTB module (CVE lookups for a known version).



Impact:

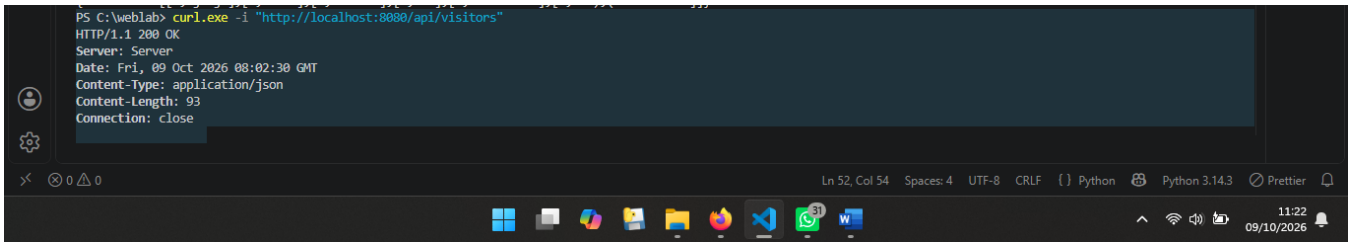
Low in isolation, but version fingerprinting is typically the first reconnaissance step in a larger attack chain, used to target version-specific known vulnerabilities.

Remediation:

Overrode Werkzeug's version-reporting attributes before starting the server:

```
WSGIRequestHandler.server_version="Server"
WSGIRequestHandler.sys_version = ""
```

Re-test



3.3 State-Changing Logic Exposed via HTTP GET

Severity: Low CVSS-style score: 3.1

Description:

The original /api/greet endpoint performed a database write (an INSERT) in response to a GET request. GET requests are expected to be safe and idempotent per HTTP semantics; using them to mutate state makes the application vulnerable to Cross-Site Request Forgery (CSRF) and means the write can be triggered by anything that causes the browser to load a URL, including a prefetch, a link preview, or a malicious <img> tag on another site.

Impact:

In the lab, this only inserts harmless rows. In a production system, the same pattern applied to a sensitive action (changing an email address, transferring funds, deleting data) is directly exploitable via CSRF.

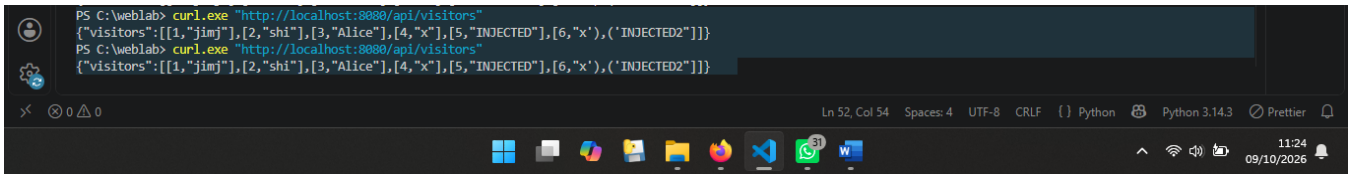
Remediation:

Split the endpoint by HTTP method: GET on /api/visitors now only reads data; writes require an explicit POST with a JSON body. /api/greet was reduced to a stateless, read-only endpoint.

```
@app.route("/api/visitors", methods=["GET", "POST"])
def visitors():
    conn = db()
    if request.method == "POST":
        name  = request.json.get("name", "stranger")
        conn.execute("INSERT INTO visitors (name) VALUES (?)", (name,))
        .....
   //Get method
```

Re-test:

Repeated GET requests to /api/visitors returned an identical, unchanged visitor list, confirming the endpoint is now read-only.



4. Overall Risk Rating

Pre-remediation, the application carried a High overall risk due to the SQL injection finding, which allowed unauthenticated, arbitrary writes to the database. Post-remediation, all identified findings were fixed and verified; residual risk is Low, consistent with a development-mode server not intended for production deployment (debug mode disabled, bound to localhost only).

5. Recommendations

- Always use parameterized queries / an ORM for any database access that includes user input — never build SQL via string concatenation or f-strings.
- Treat GET as read-only everywhere; use POST/PUT/DELETE with CSRF protection for anything that changes state.
- Strip or generalize framework version banners before any non-local deployment.
- Before moving beyond localhost, add input validation, rate limiting, and run behind a production WSGI server (not the Werkzeug dev server).
- Extend this test pass to the other OWASP Top 10 categories (XSS, CSRF tokens, auth/session handling) as the application grows.

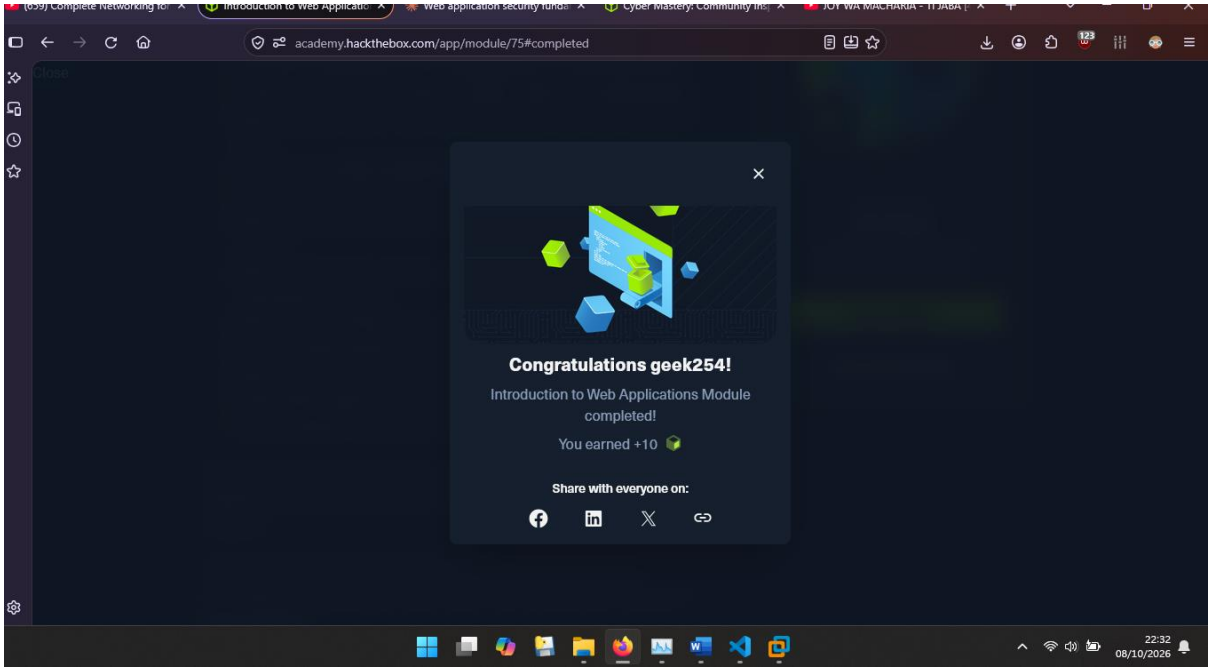
6. Appendix: Environment

Host OS	Windows 11
Runtime	Python 3.14.3, Flask, Werkzeug 3.1.9
Database	SQLite 3 (lab.db)
App location	C:\\weblab
Listener	127.0.0.1:8080 (not exposed beyond localhost)

Conclusion

This module covered the two-sided nature of web applications: a front end that is fully visible and controllable by the end user, and a back end that must never trust it. Each layer, HTML, CSS, JavaScript, web servers, application frameworks, and databases, introduces its own components and corresponding vulnerability classes, from data exposure and HTML injection on the front end to SQL injection and public CVEs on the back end. The accompanying hands-on lab put these concepts into practice end-to-end: a small application was built from scratch across every layer described above, then deliberately attacked and found vulnerable to SQL injection and a CSRF-prone GET endpoint, and finally remediated using the standard fixes for each, parameterized queries and proper HTTP method separation, with the results verified by re-testing. That build-attack-fix cycle is the practical core of this module, and the lab writeup is included below as the applied component of this report.

Lab completions proof



Links

- HTB
- linkedin